

Documentation

Caleb Kennedy Hill

Summer 2026

TODO:

- Start metrics metadata

This document is intended to serve as a reference for those of us working on the UNH Sustainability Institute's Split Incentive Tak Force.

1 Data Pipeline

This section will serve as documentation for the data pipeline. The required python libraries include `pandas`, `us`, and `census`. There are some others that are standard.

1.1 Setup

The preamble/defining involves setting the dictionaries `target_states`, `nems_munis`, and `years`. The `us.states` module takes care of the FIPS numbers for each state. Target municipalities (i.e., NEMS members) are taken from a master list; formatting them correctly is a great use for a chatbot. This output needs to be verified, though. The years of interest will depend on the application; however, one should not compare overlapping ACS5 datasets! [Click here to see why.](#)

1.2 Getting ACS5 Data

Currently there are two different types of API calls taking place: one for housing age-related data, and one for the rest. This will change.

Query helpers

Below are some useful facts about the functions used to interact with the ACS5 API.

```
acs_df_from_raw(  
    all_raw_data,  
    rename_dict=dict)
```

Returns a Pandas DataFrame from raw data dumped out of the ACS5 module of the census package: `census.acs5.state_county_subdivision()`.

```
query_states_years_variables(  
    target_states=dict,  
    target_years=list,  
    variables=dict,  
    loud=True)
```

Essentially a wrapper for `census.acs5.state_county_subdivision()`. It manually deals with some issues with a few of the municipalities. Some naming conventions aren't matching, so I've manually changed them. This happens despite GEOID being a unique identifier; it may point to a misaligned set of underlying shapefiles. [This requires further investigation.](#)

Housing Age

Several different functions exist which are intended to pull different “classes” of ACS5 variables. For instance, *Renter Landscape* (renter share, rent burden), *Housing* (structure age, structure size, household age), or *Energy* (energy burden, heating fuel). The only real difference currently is between variables related to housing age and those not related to housing age. The reason is as follows.

Far enough back in the past, ACS5 variables B25036 count numbers of structures built in a given decade. For instance, in the 2024 ACS5 B25036 counts structures built pre-1939, from 1940-1949, from 1950-1959, and so on. Things change when considering the more recent past. The 2024 ACS5 data, B25036 counts structures built 2010-2019, and then 2020-2024. On the other hand, though, in 2019, B25036 counts structures built 2010-2013, and then 2014-2019.

This creates a mismatch that would require *lots* of manual lookups and rewrites if one wanted to reuse code on different years. The functions

- `get_b25036_metadata()`
- `parse_construction_range()`
- `get_relevant_variables()`

address this mismatch, and are helpers for `get_housing_size_data()`. The helpers grab metadata for a given year, parse the metadata, and return a dictionary of appropriately-named variables for the given year. For example,

```
get_relevant_variables(2024)
{'B25036_014E': 'built_2020_to_2024',
 'B25036_015E': 'built_2010_to_2019',
 'B25036_016E': 'built_2000_to_2009',
 'B25036_017E': 'built_1990_to_1999',
 'B25036_018E': 'built_1980_to_1989',
 'B25036_019E': 'built_1970_to_1979',
 'B25036_020E': 'built_1960_to_1969',
 'B25036_021E': 'built_1950_to_1959',
 'B25036_022E': 'built_1940_to_1949',
 'B25036_023E': 'built_pre_1939'}
```

1.3 Getting GIS Data

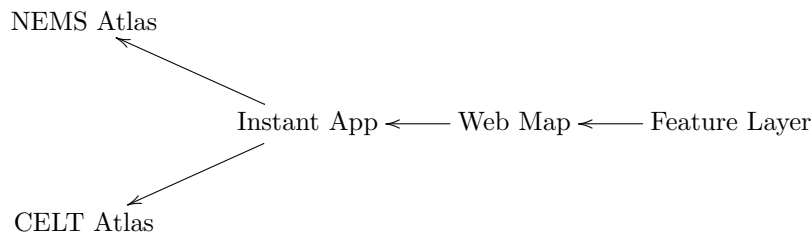
2 ArcGIS Online

ArcGIS Online (AGOL) is the platform on which we aggregate, filter, arrange, and ultimately deliver information to the NEMS Network. The basic file type for creating a map is the **shapefile**, which have suffix **.shp**. This video gives a good enough introduction to what a shapefile is. The goal of the present section is to detail the process of turning **.shp** files derived from **GeoDataFrame** into ArcGIS Instant Apps, for use in Portfolio projects. The minimal system schematic is as follows:

Portfolio $\leftarrow$ Instant App $\leftarrow$ Web Map $\leftarrow$ Feature Layer

For purposes, a **Portfolio** is a collection of Instant Apps, and a mini-browser to view them in. We will be tying into the efforts of and extending the work of BU's CELT program. With this in mind, our prototypical example of a Portfolio should be their Legacy Transition Atlas, which is linked here.

The AGOL platform is quite modular. Once it is created, an Instant Apps can be specified just by URL. This means that any products we deliver can be displayed anywhere else. That is to say, we may have our own NEMS Portfolio existing in parallel to the CELT Portfolio. It might be something like:



Once we get around to actually creating a Portfolio, we will see that the diagrams might be more complex. For now, the simple one-line diagram should be a sufficient mental model.

Style Considerations

In order to return standardized products, adhere to the following style/color standards. **These are an invention of me, Caleb Kennedy Hill, and are by no means “correct.”** But it's what we're doing so far.

Some data are transformed from continuous numbers to classes. E.g., 1.369 is “low,” 6.893 is “medium,” and 10.733 is “high.” For cases when AGOL is able to do this automatically, the color scheme **Purple 1** should be used for NEMS members, and **Green 5** for non-NEMS members. When AGOL cannot automatically do this, use the following table for colors:

	Low	Medium	High
NEMS	# eecec9	# d588b9	# a14e96
non-NEMS	# e2ebd3	# add46a	# 2d8078

For example, in the Housing Size map, we translate Low, Medium, and High to mean 1, 2-4, and 5+ units, respectively.

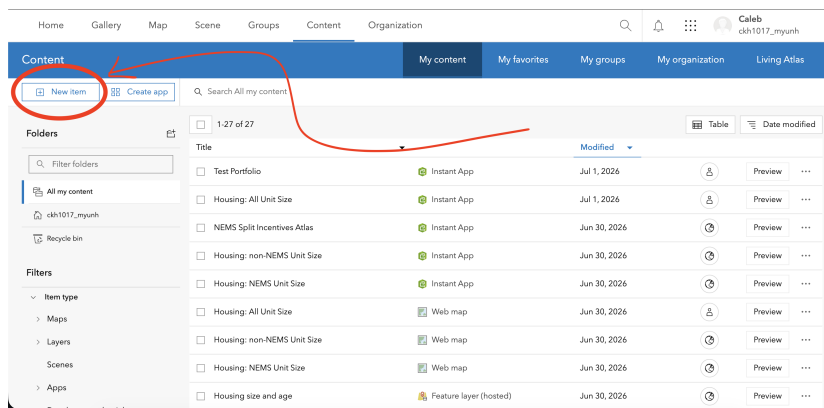
Create an Instant App

Here is where we will create the Instant Apps which will be linked inside the Portfolio.

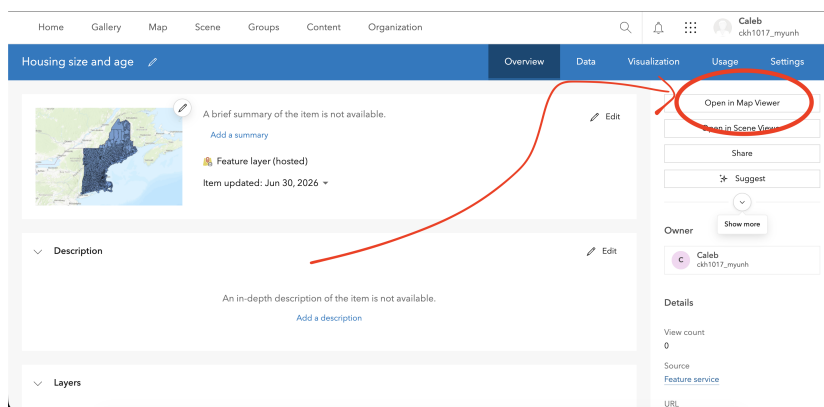
IMPORTANT: To do any of this you will need an AGOL account with publishing permissions. UNH has an AGOL license, but we need to contact admin to get permissions.

1. Prepare ShapeFiles. Be sure that the relevant **GeoDataFrame** has been exported and compressed into a **.zip** file. The process of obtaining the shapefiles is outlined in Section 1.
2. Upload the **.zip** file to AGOL as a **Feature Layer**. Navigate to the **Content** tab of AGOL, then click **New item**. Click the **Feature layer**, and then the **Upload a file** options. Be sure to choose the upload option that creates a Hosted Feature Layer. ¹ Pick a good name, and add a helpful description.

¹This step is not possible without publishing permissions!



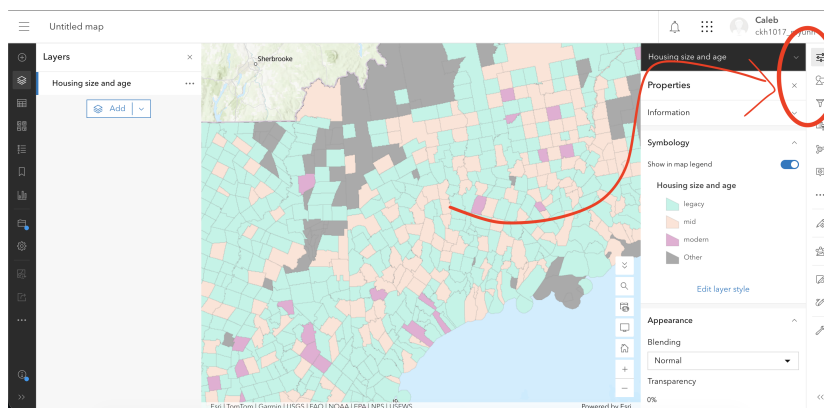
3. Back in the **Content** window, open the Hosted Feature Layer you just created. Choose the option to **Open in Map Viewer** in the upper right.



4. Fiddle and Save. The righthand side of the screen has a menu for changing how the map looks. We will almost exclusively tweak the top three options: Properties, Style, and Filters. Here are some style notes which may be standardized:

- 0% transparency
- Continuous variables classed into three quantiles
- NEMS members are colored Purple 1, and non-NEMS are colored Green 5. If colors need to be copied exactly, they can be obtained by opening another map and inspecting the style options.

Now is when the NEMS/non-NEMS filter should be applied, i.e., filter for `nems=1` or `nems=0`.

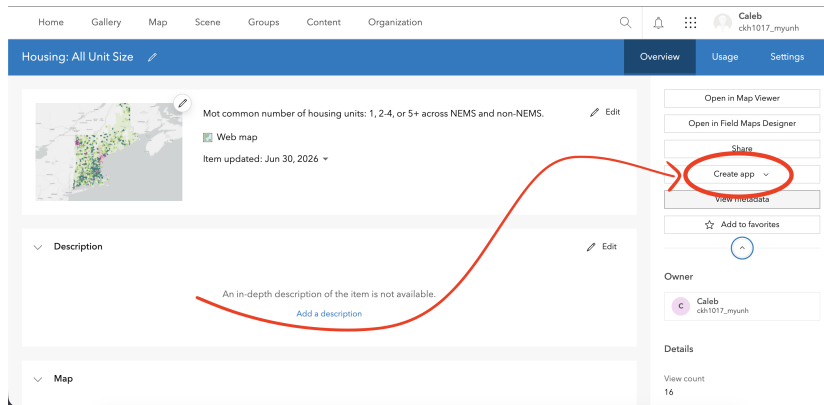


When you're done, click **Save** as from the lefthand menu and pick the same name you used for the Feature Layer; add a short description as well.

NOTE: For almost every data point, we are delivering three different views: NEMS, non-NEMS, and all. Thus we need to create one Web Map with the filter set to `nems=1`, one Web

Map with **nems=0**, and one with no filter on that field. Repeat all previous steps for each of these maps.

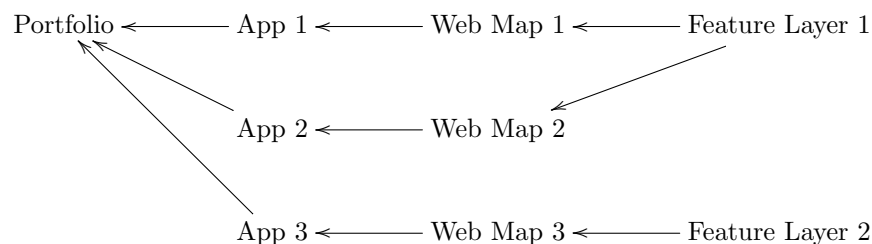
5. In the **Content** window, open Web Map you just created. In the options on the right, click **Create app** and choose **Instant App**. In the page that opens, choose the **Basic (Media Map)** option. Name it the same thing as the previous two. Not much needs to change here.



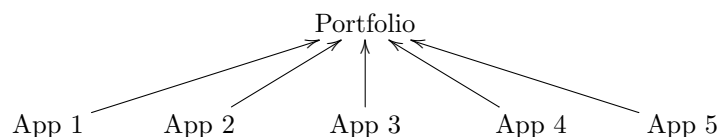
Thus you have created an Instant App! This app can be shared directly via its URL. Its default visibility is set to Private, meaning only the owner can view it. This can be changed at the current step, or later when publishing a Portfolio.

Create a Portfolio

A portfolio can house several different Instant Apps:



Fortunately, once it's time to create a Portfolio, there's not much need to look past the second layer. So in reality our working model should be something like:



1. In the Content tab, click the button to **Create app**. Choose the **Portfolio** option, then choose a good name.
2. In the screen that opens next, click on **Step 1. Portfolio**. In the **App layout** options, choose **Accordion**.
3. Now click the **+** sign to add a new section, then click **Browse for content**. There is a Web Map called Base New England owned by me (username **ckh1017_myunh**) which should serve as a nice blank opening screen. Search for this Web Map in the search bar at the top of the screen that opens; be sure to not just search your own content, but that of the organization.
4. Now you should be in the editing screen for that new section. Change the name to something relevant, then click **Edit description**. The description is where we will hyperlink to the existing Instant Apps.

5. Add a few words describing the apps you intend to display in this section. Then use the hyperlink button or **Ctrl/Cmd+K** and paste the URL of the desired Instant App.²
6. Apply the following settings:
 - Open in Map Viewer link : off
 - Terms of use : off
 - Open links in app : on **IMPORTANT**
 - Use item details : off
7. Finish editing the section by clicking **Done**. Either add more sections or publish the Portfolio. Again, make sure share settings are set to public.

²AGOL has a very annoying bug of opening the window to paste a hyperlink *behind* the description editing window. Most of the time I've been able to work around this.